

Evaluación de estrategias de chunking para sistemas RAG ligeros: calidad de recuperación y costo computacional

Renzo Fernando Mosquera Daza y Luna Catalina Martínez Vásquez

Universidad ICESI · Curso de Inteligencia Artificial

Renzo.mosquera@u.icesi.edu.co · Luna.martinez@u.icesi.edu.co

Resumen—Este trabajo evalúa tres estrategias de chunking para recuperación aumentada por generación sobre QASPER: fixed-size, semantic breakpoint y structure-aware, una propuesta que utiliza límites de párrafo y un tamaño máximo de fragmento. La ejecución final procesó 100 documentos del conjunto de validación y 250 preguntas con evidencia mapeada mediante el modelo all-mpnet-base-v2. Fixed-size alcanzó $F1@5=0,0515$ y structure-aware $F1@5=0,0505$, mientras semantic breakpoint obtuvo 0,0302. Structure-aware logró el mayor $F1@10$ (0,0672) y el menor costo: 860,4 s de preprocesamiento y 492.023 tokens efectivos, frente a 1.019,7 s/572.404 tokens de fixed-size y 1.211,6 s/711.032 tokens de semantic breakpoint. Los resultados indican que la estructura documental puede conservar una calidad competitiva reduciendo el trabajo de preprocesamiento.

Índice de términos—chunking, RAG, QASPER, recuperación de evidencia, embeddings, costo computacional, structure-aware.

I. INTRODUCCIÓN

Los sistemas Retrieval-Augmented Generation (RAG) combinan generación con información recuperada desde una colección externa, por lo que la forma de dividir documentos extensos en chunks influye en el contexto disponible para la búsqueda y la generación [2]. Fixed-size chunking es simple y barato, pero puede separar ideas relacionadas; semantic chunking intenta preservar coherencia usando representaciones vectoriales, aunque añade procesamiento. Qu, Tu y Bao [3] mostraron que ese costo adicional no produce mejoras consistentes en todos los escenarios. A partir de ese resultado, este trabajo propone una extensión orientada a documentos científicos: structure-aware chunking, que usa párrafos y un tamaño máximo como señales de segmentación. El objetivo es comparar fixed-size, semantic breakpoint y structure-aware bajo el mismo pipeline, observando no solo Precision, Recall y F1, sino también tiempo, tokens y comportamiento de los chunks.

II. MATERIALES Y MÉTODOS

A. Diseño del estudio

Se realizó una replicación parcial y una extensión del estudio base [3], limitada a recuperación de evidencia. Las tres estrategias se evaluaron con el mismo modelo de embeddings, los mismos documentos, las mismas preguntas y semilla 42, de forma que el criterio de chunking fuera la variable principal.

B. Conjunto de datos

Se utilizó QASPER, un conjunto de preguntas y respuestas ancladas en artículos científicos [1]. La ejecución final tomó 100 documentos del split validation y 250 preguntas con al menos una evidencia mapeada. La carga se realizó mediante la conversión oficial en Parquet disponible a través de Hugging Face Datasets [6].

C. Preprocesamiento y representación

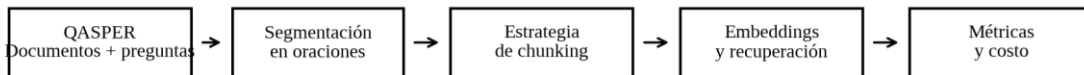
Cada artículo se reconstruyó conservando secciones y párrafos y luego se segmentó en oraciones con el sentencizador de spaCy [7]. Preguntas y chunks se codificaron con SentenceTransformers [4] usando all-mpnet-base-v2, basado en MPNet [5]. Los vectores se normalizaron y se compararon mediante producto punto; el límite de secuencia del modelo fue de 384 tokens.

D. Estrategias de chunking

Fixed-size. Agrupa seis oraciones consecutivas con solapamiento de una oración. **Semantic breakpoint.** Calcula embeddings de las oraciones, mide la distancia semántica entre pares consecutivos y selecciona los siete mayores cambios para formar aproximadamente ocho chunks por documento. **Structure-aware.** Respeta los límites de párrafo; si un párrafo supera ocho oraciones, lo divide internamente con solapamiento de una oración.

TABLA I
COMPARACIÓN CONCEPTUAL

Estrategia	Señal de corte	Costo del corte
Fixed-size	Posición / tamaño	Bajo
Semantic breakpoint	Distancia semántica	Alto
Structure-aware	Párrafo + máximo	Bajo



Fixed-size | Semantic breakpoint | Structure-aware

Fig. 1. Flujo experimental empleado para comparar calidad de recuperación y costo computacional.

E. Recuperación y métricas

Para cada pregunta se recuperaron los k chunks con mayor similitud, con $k \in \{1,3,5,10\}$. Los chunks recuperados se mapearon a sus oraciones y se compararon con las evidencias de referencia. Precision representa qué fracción de lo recuperado corresponde a evidencia; Recall, qué fracción de la evidencia disponible fue localizada; y F1 resume el equilibrio entre ambas. La comparación principal usa F1@5, manteniendo F1@10 para observar el efecto de ampliar la profundidad de búsqueda.

F. Medición del costo

Se midieron el tiempo de construcción de chunks, el tiempo de embeddings del índice, los tokens efectivos procesados, el número de chunks, el tamaño medio y los fragmentos truncados por el límite del modelo. En semantic breakpoint también se contabilizaron los tokens empleados para decidir los cortes. Los tiempos solo se comparan dentro de esta ejecución y hardware.

III. RESULTADOS

Fixed-size obtuvo el mayor F1@5 (0,0515), seguido muy de cerca por structure-aware (0,0505). Semantic breakpoint alcanzó mayor Recall@5, pero su baja Precision@5 redujo el F1. Al ampliar la búsqueda a $k=10$, structure-aware obtuvo el mayor F1 (0,0672).

El comportamiento al variar k también fue distinto entre estrategias. Fixed-size creció hasta $k=5$ y luego descendió ligeramente; semantic breakpoint alcanzó su mejor F1 en $k=3$ y disminuyó al recuperar más chunks. Structure-aware, en cambio, aumentó de forma sostenida desde F1@1=0,0343 hasta F1@10=0,0672, lo que indica que su fragmentación más fina aprovecha mejor una recuperación más profunda.

TABLA II
MÉTRICAS DE RECUPERACIÓN

Estrategia	P@5	R@5	F1@5	F1@10
Fixed-size	0,0388	0,1108	0,0515	0,0471
Semantic	0,0175	0,1629	0,0302	0,0248
Structure	0,0473	0,0702	0,0505	0,0672

TABLA III
COSTO DE PREPROCESAMIENTO

Estrategia	Chunks	Tokens efect.	Tiempo (s)	Trunc.
Fixed-size	3.110	572.404	1.019,7	36
Semantic	800	711.032	1.211,6	385
Structure	4.672	492.023	860,4	7

Structure-aware procesó 14,0 % menos tokens y requirió 15,6 % menos tiempo total que fixed-size. Frente a semantic breakpoint, las reducciones fueron de 30,8 % en tokens y 29,0 % en tiempo. Además, presentó solo siete chunks truncados, frente a 36 y 385, respectivamente.

El contexto medio recuperado en top-5 fue de 341,6 tokens para structure-aware, 822,7 para fixed-size y 966,7 para semantic breakpoint. Este resultado muestra que la propuesta recuperó contextos más compactos, aunque con menor Recall@5 que las otras estrategias.

El tamaño efectivo promedio por chunk fue de 184,1 tokens en fixed-size, 253,7 en semantic breakpoint y 105,3 en structure-aware. En el percentil 95, los tamaños fueron 295, 384 y 231 tokens, respectivamente. La concentración de semantic breakpoint en el límite de 384 tokens coincide con su elevado número de truncamientos.

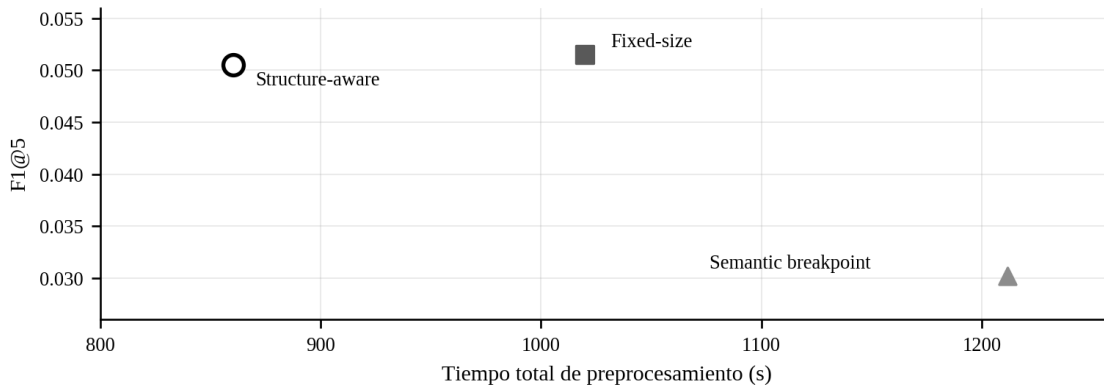


Fig. 2. F1@5 frente al tiempo total de preprocesamiento. Structure-aware mantiene un F1 cercano a fixed-size con menor costo temporal.

IV. DISCUSIÓN

Los resultados respaldan parcialmente la hipótesis de que la estructura explícita de los artículos puede ser una señal útil de segmentación. Structure-aware no superó a fixed-size en F1@5, pero la diferencia absoluta fue de solo 0,0009, mientras redujo 14,0 % los tokens y 15,6 % el tiempo. Al recuperar diez chunks, la propuesta sí obtuvo el mejor F1, con una mejora relativa de 42,5 % frente a fixed-size. Por tanto, la ventaja observada no consiste en maximizar una sola métrica, sino en conservar calidad competitiva con menor costo y escalar mejor cuando aumenta k.

Semantic breakpoint produjo únicamente 800 chunks, con un promedio de 19,3 oraciones, y 385 excedieron el límite de 384 tokens. Su Recall@5 fue el mayor, pero la Precision@5 fue la menor. Este patrón es compatible con chunks extensos que incluyen evidencia junto con más contenido irrelevante. Structure-aware generó 4.672 chunks más pequeños, con un promedio de 3,34 oraciones, siete truncamientos y el contexto top-5 más compacto. La fragmentación adicional favoreció la precisión, aunque a k=5 redujo el recall; a k=10 esa limitación disminuyó.

Implicaciones de costo

El costo adicional del enfoque semántico aparece antes de indexar los chunks: semantic breakpoint procesó 508.111 tokens de oraciones únicamente para decidir dónde cortar, además de 202.921 tokens efectivos de indexación. Structure-aware no requiere esa etapa de embeddings para segmentar; usa información estructural que ya está presente en el documento. En esta corrida, esa diferencia redujo tanto el tiempo total como el volumen de tokens procesados sin deteriorar el F1 frente al enfoque semántico.

Limitaciones

El estudio utiliza 100 documentos del split validation y una muestra de 250 preguntas, no la totalidad de QASPER. El mapeo de evidencia a oraciones combina coincidencia directa y similitud aproximada, y los hiperparámetros se fijaron sin una búsqueda exhaustiva. Además, los tiempos dependen del hardware utilizado. Estas condiciones limitan la generalización de los valores absolutos, aunque no invalidan la comparación controlada dentro de la misma ejecución.

V. CONCLUSIONES

La evaluación muestra que structure-aware ofrece el compromiso más favorable entre calidad y costo dentro de la configuración estudiada. Su F1@5 quedó prácticamente empatado con fixed-size y superó a las otras estrategias en F1@10, mientras presentó el menor tiempo, el menor consumo de tokens y el menor número de truncamientos. Estos resultados sugieren que, en artículos científicos, aprovechar límites de párrafo puede evitar parte del costo de decidir cortes con embeddings semánticos sin sacrificar de forma importante la recuperación. En consecuencia, la estructura documental constituye una alternativa práctica para sistemas RAG ligeros cuando el costo de preprocesamiento es relevante.

Trabajo futuro y reproducibilidad

Como trabajo futuro se propone extender la evaluación al split test completo de QASPER, explorar otros tamaños máximos y solapamientos, y añadir generación de respuestas para medir el efecto de los chunks sobre la etapa generativa. También sería útil repetir la medición en hardware controlado y con más de un modelo de embeddings para separar el efecto del chunker del efecto de la representación. El notebook, la landing interactiva y los artefactos de resultados se encuentran en el repositorio del proyecto [8].

CONFIGURACIÓN FINAL

Parámetro	Valor
Documentos / preguntas	100 / 250
Split / semilla	validation / 42
Modelo	all-mpnet-base-v2
Máx. secuencia	384 tokens
k evaluados	1, 3, 5, 10

REFERENCIAS

- [1] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith y M. Gardner, “A Dataset of Information-Seeking Questions and Answers Anchored in Research Papers,” en Proc. NAACL-HLT, 2021, pp. 4599–4610, doi: 10.18653/v1/2021.naacl-main.365.
- [2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” en Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.
- [3] R. Qu, R. Tu y F. S. Bao, “Is Semantic Chunking Worth the Computational Cost?,” en Findings of the Association for Computational Linguistics: NAACL 2025, pp. 2155–2177, 2025, doi: 10.18653/v1/2025.findings-naacl.114.
- [4] N. Reimers y I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” en Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.
- [5] K. Song, X. Tan, T. Qin, J. Lu y T.-Y. Liu, “MPNet: Masked and Permuted Pre-training for Language Understanding,” en Advances in Neural Information Processing Systems, vol. 33, 2020.
- [6] Q. Lhoest et al., “Datasets: A Community Library for Natural Language Processing,” en Proc. EMNLP: System Demonstrations, 2021, pp. 175–184, doi: 10.18653/v1/2021.emnlp-demo.21.
- [7] M. Honnibal, I. Montani, S. Van Landeghem y A. Boyd, “spaCy: Industrial-strength Natural Language Processing in Python,” Explosion, 2020. [En línea]. Disponible: <https://spacy.io/>.
- [8] R. F. Mosquera Daza y L. C. Martínez Vásquez, “chunking_rag_interactivo,” repositorio de código y resultados, 2026. [En línea]. Disponible: https://github.com/RenzoFernando/chunking_rag_interactivo